UNIVERSIDAD MARIANO GÁLVEZ DE GUATEMALA

Facultad de Ingeniería en Sistemas de Información

Desarrollo de Aplicaciones Web

FerreSync

Sistema de punto de venta web distribuido para ferretería

Proyecto Final No. 1 · PHP, MariaDB, Docker y API REST

Estudiante y carné: completar antes de la entrega.

Docente: M.Sc. Elmer Rolando Lux Laynez

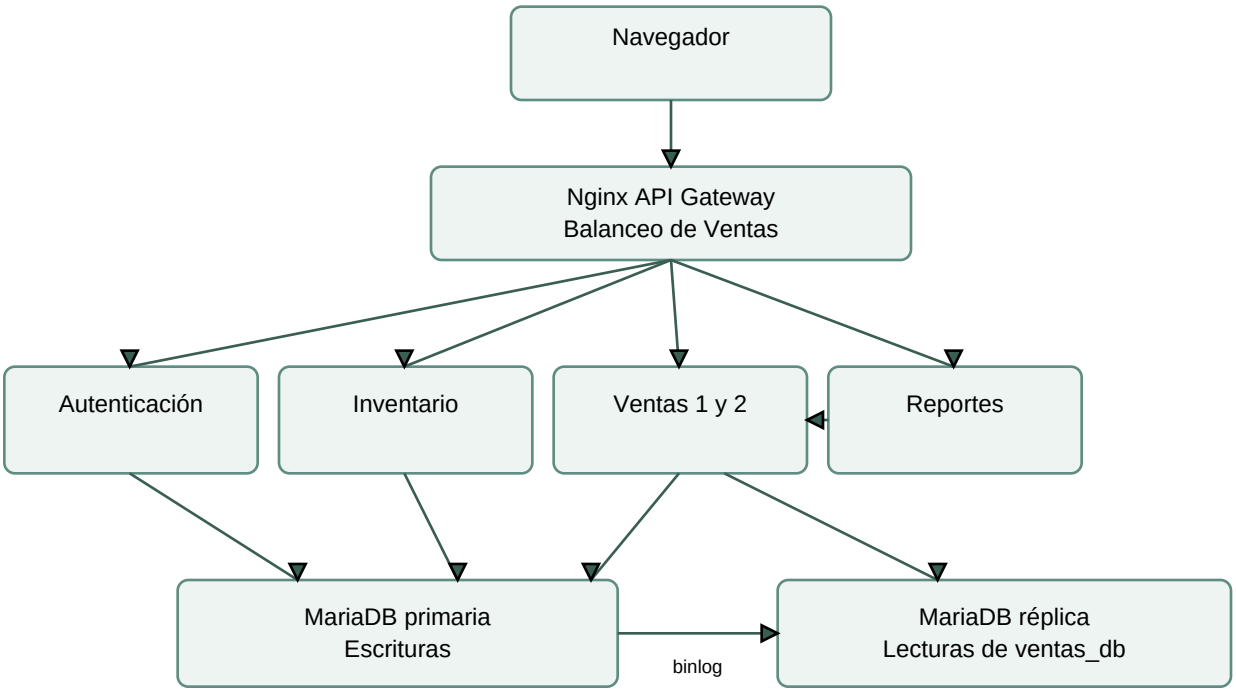
Fecha de revisión técnica: 23 de octubre de 2026

Entrega indicada en el enunciado: 31 de octubre de 2026

Este documento explica la arquitectura implementada, el modelo de datos, los permisos y las pruebas locales. La operación en efectivo está validada. El pago Sandbox, la consulta real del clima y la publicación externa permanecen pendientes; el documento debe actualizarse al completarlos.

Arquitectura distribuida

Nginx es la entrada del navegador. Autenticación, Inventario, Ventas y Reportes son procesos PHP independientes. Dos contenedores ejecutan Ventas para repartir peticiones; ambos representan el mismo microservicio y comparten sus datos.



Ventas llama a Inventario por REST para descontar o devolver existencias y verifica el JWT con Autenticación. Reportes obtiene estadísticas por REST de Ventas; Ventas consulta su propia base en la réplica. Ningún servicio lee tablas de otro servicio directamente.

El balanceo tolera la caída de una instancia de Ventas. No implica failover de MariaDB ni elimina las dependencias únicas de Autenticación, Inventario y Nginx. Las claves JWT y credenciales no se incluyen en las imágenes ni en el repositorio.

Modelo de datos y consistencia

Base	Tablas y responsabilidad
auth_db	usuarios y usuario_eventos: credenciales bcrypt, rol, estado y cambios de acceso.
inventario_db	categorias, productos, movimientos_stock, descuentos_venta y devoluciones_venta.
ventas_db	clientes, ventas, detalle_ventas, pagos y venta_eventos.
reportes_db	Reservada. Reportes obtiene los datos por REST; no duplica las ventas.

Relaciones de Ventas: un cliente identificado puede tener muchas ventas; cada venta tiene sus detalles, un cobro y varios eventos administrativos. Las relaciones internas tienen claves foráneas. Los identificadores de producto y vendedor no tienen una FK a otra base de microservicio.

Historia del comprobante: ventas conserva una copia del nombre, NIT y dirección del comprador. detalle_ventas conserva código, nombre, categoría y precio del producto. Modificar un cliente o producto no altera comprobantes anteriores.

Efectivo: pagos registra importe, moneda, método, proveedor y estado. Actualmente se valida el cobro de caja; no se guardan datos de tarjetas. Una anulación marca también el cobro en efectivo como anulado.

Importes: precios en quetzales con IVA incluido del 12 %. Los cálculos de caja utilizan centavos enteros; la interfaz muestra Q y dos decimales. Desde Q 2,500.00 se exige NIT, nombre y dirección. La caja trabaja con cantidades enteras.

Replicación: server_id 1 para primaria y 2 para réplica, binlog ROW y filtro ventas_db. Reportes usa una cuenta SELECT. La réplica no sustituye un respaldo: los cambios y eliminaciones también se copian.

Roles y cambios administrativos

Operación	Administrador	Cajero
Consultar inventario y alertas	Sí	Sí
Gestionar productos y categorías	Sí	No
Registrar ventas y ver facturas propias	Sí	Sí
Consultar ventas de otros vendedores	Sí	No
Corregir comprador y anular	Sí	No
Reportes y exportación PDF	Sí	No
Registrar usuarios y cambiar acceso ajeno	Sí	No

Los permisos de los dos roles son fijos en PHP. El administrador puede cambiar el rol y estado de otras cuentas desde Usuarios; no puede cambiar su propio acceso. Cada verificación REST vuelve a consultar el rol vigente, de modo que un JWT anterior no mantiene privilegios retirados.

Corrección: solo nombre, NIT y dirección del comprador, con motivo obligatorio. Para cambiar productos o cantidades se anula la venta y se registra otra. El registro administrativo conserva autor, fecha y datos anteriores/posteriores.

Anulación: la venta pasa a anulando antes de llamar a Inventario. Inventario devuelve las cantidades del descuento original una sola vez. Al confirmar la devolución, Ventas marca anulada. Si falla la comunicación, un reintento termina la operación sin duplicar existencias.

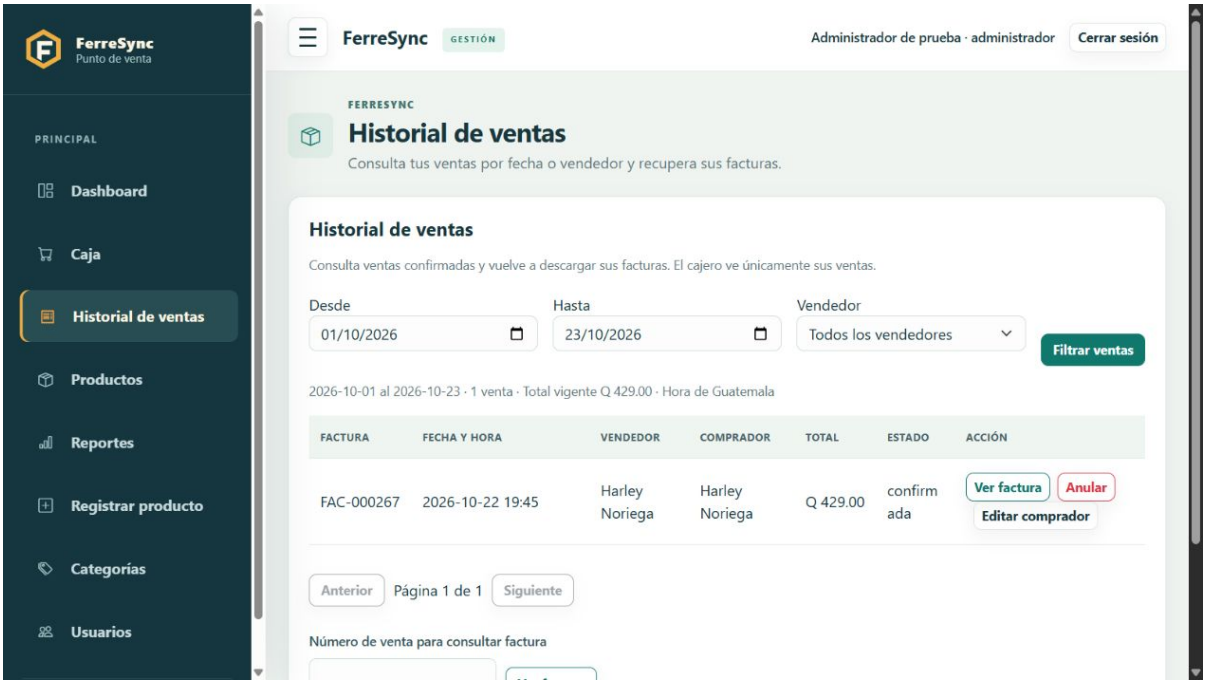
La venta anulada conserva su número, puede consultarse y su PDF indica COMPROBANTE ANULADO. Ya no suma en reportes. FerreSync genera comprobantes académicos sin certificación FEL; no anula documentos fiscales en SAT.

API REST principal

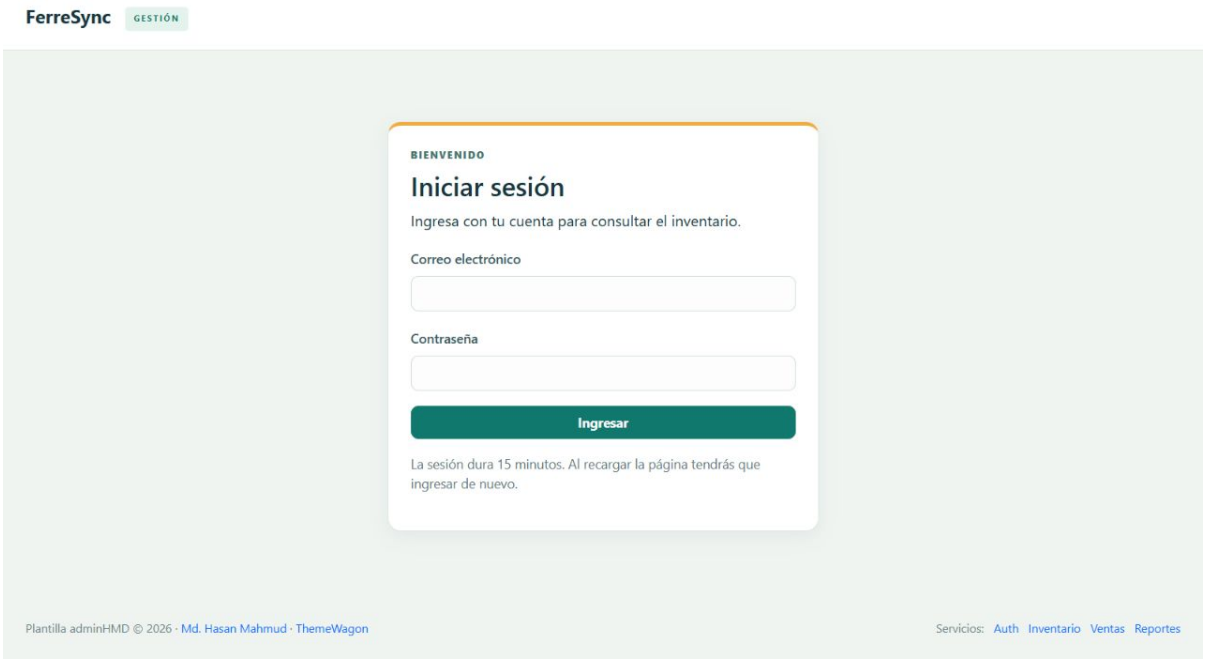
Método	Ruta	Responsabilidad
POST	/api/auth/login	Credenciales a JWT
POST	/api/auth/register	Alta de usuario, admin
POST	/api/auth/verify	Verificación de identidad
GET / PATCH	/api/auth/users / users/{id}	Consulta y cambio de acceso, admin
GET / POST	/api/inventario/productos	Consulta o alta
PUT / DELETE	/api/inventario/productos/{id}	Modificar o desactivar, admin
GET	/api/inventario/alertas	Existencias menores que 5
POST / GET	/api/ventas/	Registrar o consultar historial
GET / PATCH	/api/ventas/{id}	Consulta o corrección administrativa
POST	/api/ventas/{id}/anular	Anulación, solo admin
GET	/api/ventas/{id}/eventos	Auditoría, solo admin
GET	/api/ventas/{id}/factura/pdf	Comprobante PDF protegido
GET	/api/ventas/{id}/factura/qr	Imagen QR protegida
GET	/api/reportes/ventas	Período, serie y listado
GET	/api/reportes/top-productos	Productos más vendidos
GET	/api/reportes/pdf	Exportar estadísticas
GET	/api/reportes/clima	Clima o configuración pendiente

Las rutas protegidas reciben Authorization: Bearer JWT. Altas y cambios usan JSON, salvo la imagen del producto que utiliza multipart. Los errores devuelven error y un código HTTP: 401 sin acceso válido, 403 sin permiso, 404 no encontrado, 409 conflicto, 422 validación y 503 dependencia no disponible. El catálogo completo está en docs/endpoints.md.

Interfaz y recorrido del usuario



Historial verificado en el navegador: filtros por vendedor y fecha, estado del comprobante y acciones administrativas. El cajero consulta únicamente sus propias ventas y no recibe botones para corregir o anular.



La interfaz utiliza AdminHMD y Bootstrap con archivos locales. Se conserva el crédito de la plantilla y se adapta la navegación a los módulos y permisos de FerreSync. La captura corresponde al diseño aplicado en el bloque de interfaz.

Instalación, pruebas y pendientes

Primera instalación: Docker con Compose v2 y PowerShell 7.4+. Ejecutar pwsh -File scripts/start.ps1 -Tools. Se preparan variables locales, claves JWT, migraciones y réplica. El administrador inicial se crea mediante la utilidad interactiva documentada; después se registra el cajero desde Usuarios.

Arranque posterior: docker compose up --build -d y recargar Nginx si se recrearon servicios. No usar down -v para actualizar. La preparación desde bases vacías se verificó en un proyecto Docker aislado y puerto 18080, sin usar las bases reales.

Verificación	Resultado local
Nuevo bloque de cambios y acceso	42 comprobaciones
Ventas y factura	79 comprobaciones
Reportes	55 comprobaciones
Historial	29 comprobaciones
Contrato del clima sin clave real	3 comprobaciones
Balanceo	Dos instancias; continuidad con una detenida
Instalación desde cero	Claves, migraciones, réplica y cuatro /health correctos

Pendientes para la entrega completa: integrar y probar Stripe o PayPal Sandbox; cargar una clave OpenWeatherMap y validar datos reales; crear/publicar el repositorio GitHub; desplegar en Hostinger con URL pública y TLS; confirmar con el docente la aceptación del VPS frente al hosting gratuito indicado. Completar datos del estudiante y actualizar este documento después de las pruebas externas.

Fuentes técnicas oficiales: documentación de MariaDB sobre replicación; Nginx sobre HTTP load balancing; OpenWeatherMap Current Weather. Las comprobaciones del sistema se encuentran en scripts/ y tests/. El guion de diez minutos está en docs/entrega/guion-demo.md.